

Pre-trained 모델 활용

04 텍스트 분류 모델

목차

—
텍스트 분류 모델이
무엇인지 알아보고 감정
분류 모델을 사용하는
방법을 배워봅니다.

01 텍스트 데이터 전처리

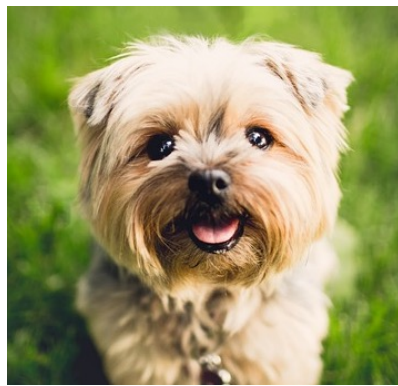
02 텍스트 분류 모델

01

텍스트 데이터 전처리

☑ 텍스트 분류 모델

이미지 분류 모델



모델



"강아지"

텍스트 분류 모델

"오늘은 기분이 좋지 못한 날입니다."



모델



"부정적"

④ 텍스트 분류 모델

텍스트를 입력받고 어디에 포함되는지 예측하는 모델

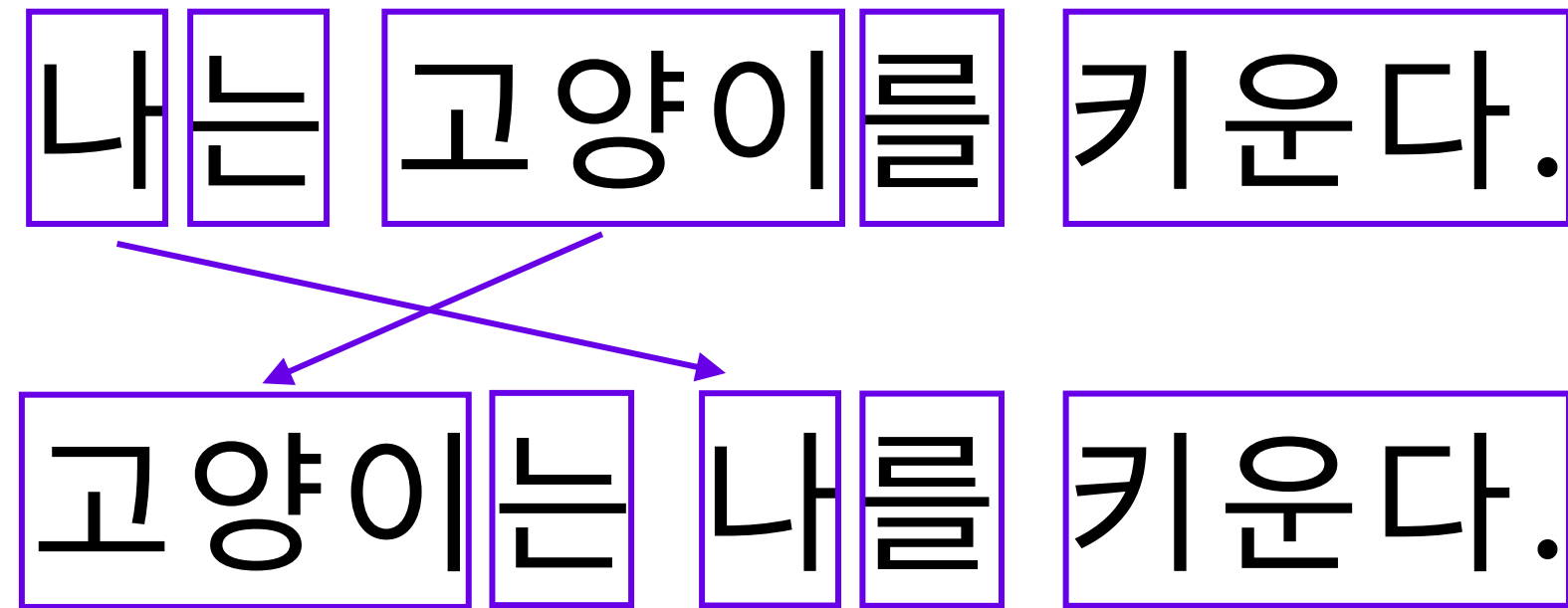
- 텍스트를 입력으로 받는 모델
- 출력은 이 텍스트가 속하는 클래스를 예측한 결과

④ 텍스트 데이터의 특징

“안녕하세요”
“오늘은 기분이 좋지 못한 날입니다.”
“I am a boy”

- 길이가 다양함
- 단어나 문장이 특정한 순서로 나란히 배열되어 있음
- 이 순서에 따라 데이터의 의미가 달라짐

④ 시퀀스 데이터(sequence data)



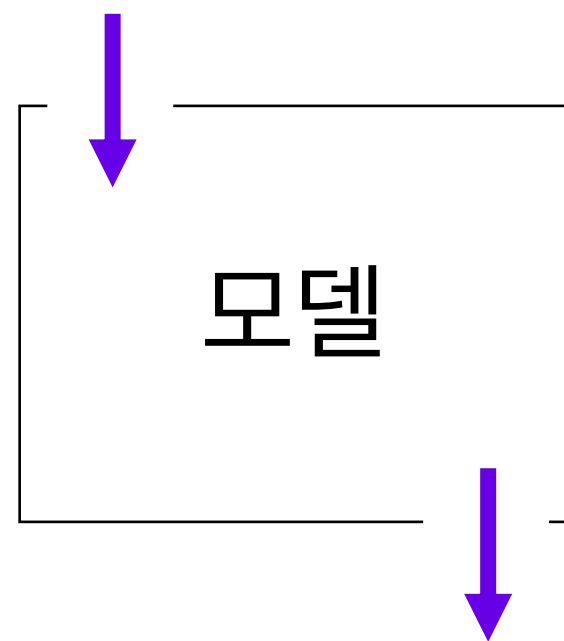
- 순서가 있는 데이터
- 순서를 바꾸면 다른 의미가 되는 데이터

④ 자연어 데이터의 전처리

"오늘은 기분이 좋지 못한 날입니다."

데이터 전처리 과정

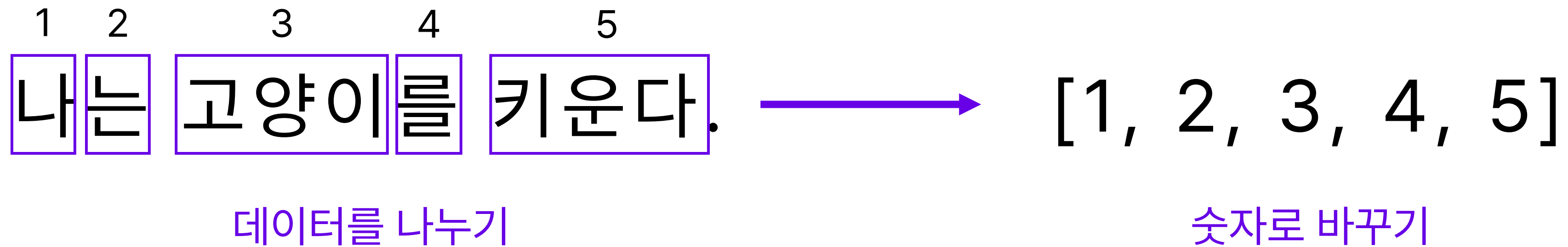
(숫자들의 행렬)



"부정적"

- 숫자들의 행렬만 입력받는 모델
- 데이터를 숫자의 행렬로 변환하는 과정이 전처리
- 이미지 데이터와는 다른 방법으로 전처리가 필요함

④ 자연어 데이터의 전처리 과정



1. 자연어는 문자, 단어, 문장 등의 배열이기 때문에 특정 단위로 데이터를 나누어 처리
2. 나누어진 요소를 전처리 과정을 통해 숫자로 바꿈

④ 토큰화(Tokenization)

주어진 텍스트를 토큰이라는 단위로 나누는 작업

- 토큰은 단어, 형태소 등 의미가 있는 단위로 정의함.
- 토큰을 나눴다면 각 토큰에 번호를 붙여 숫자형 데이터로 변환할 수 있음

01 텍스트 데이터 전처리

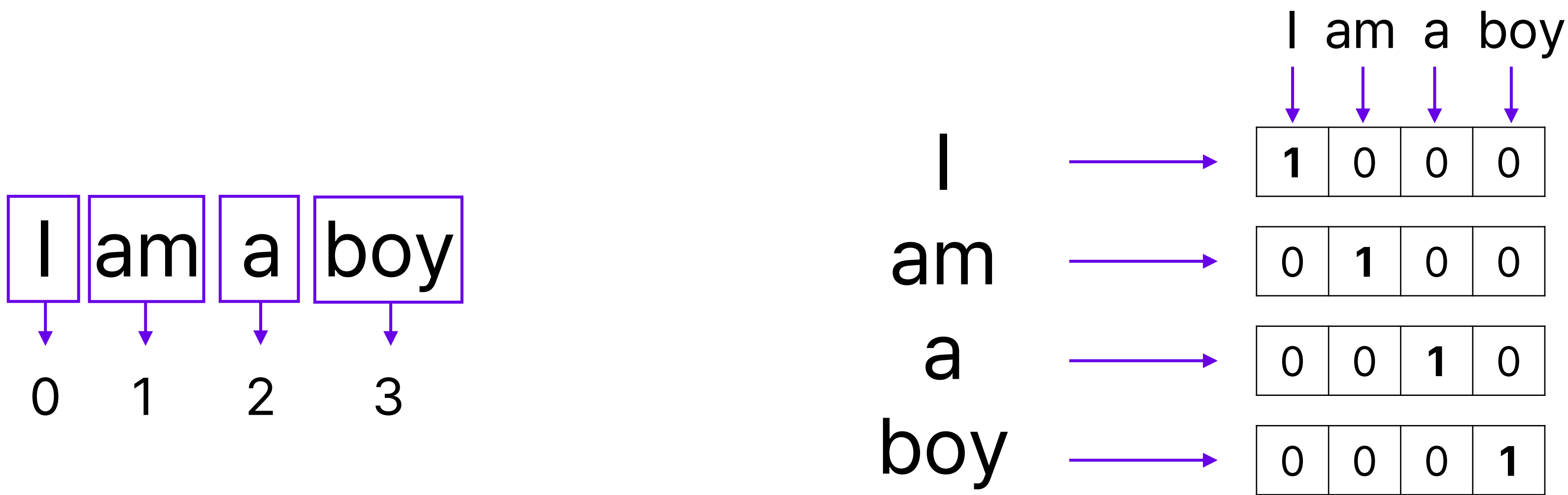
☑ 단어 토큰화

I am a boy

- 단어 단위로 자연어 데이터를 나누는 방법

④ 원-핫 인코딩(One-hot encoding)

하나만 1이고 나머지가 0인 행렬로 각 단어를 표현



④ 원-핫 인코딩의 한계

단어의 개수가 늘어날 수록 필요한 저장 공간이 늘어남

- 1000개의 단어가 있다면 999개의 0과 1개의 1로 구성된 벡터가 하나의 단어를 표현함
- 공간 측면에서 매우 비효율적인 방법

④ 원-핫 인코딩의 한계

단어의 유사도를 표현하지 못함

- 각 칸이 어떤 단어의 자리인지 모르면 행렬만 가지고는 두 문장의 유사도를 표현할 수 없음.
- 비효율적이고 단순한 매핑이라는 것을 의미함.

④ 단어 임베딩(word embedding)

유사성을 반영해 정해진 길이의 배열로 행렬을 압축

- 단어 간 유사성을 계산하여 표현
- 단어가 많아지더라도 행렬의 길이가 길어지지 않음
- 많은 길이를 할당할수록 더 높은 정확도로 표현

02

텍스트 분류 모델

④ 텍스트 분류 모델의 예시

텍스트를 입력하면 어떤 종류의 내용인지 예측하는 모델

- 메일의 내용을 보고 스팸인지 아닌지 분류하는 모델 -> 스팸인지 아닌지 2종류
 - 주어진 글을 보고 부정적인 글인지 아닌지 분류하는 모델 -> 긍정인지 아닌지 2종류
 - 뉴스 기사를 보고 어떤 주제의 기사인지 분류하는 모델 -> 경제, 시사, 정치 등등 3종류 이상
- ...

④ 이진 분류 모델

두 가지 중 어디에 속하는지 예측하는 모델

- 종류가 두가지밖에 없는 경우 사용
- 맞으면 1, 아니면 0 를 출력하는 모델

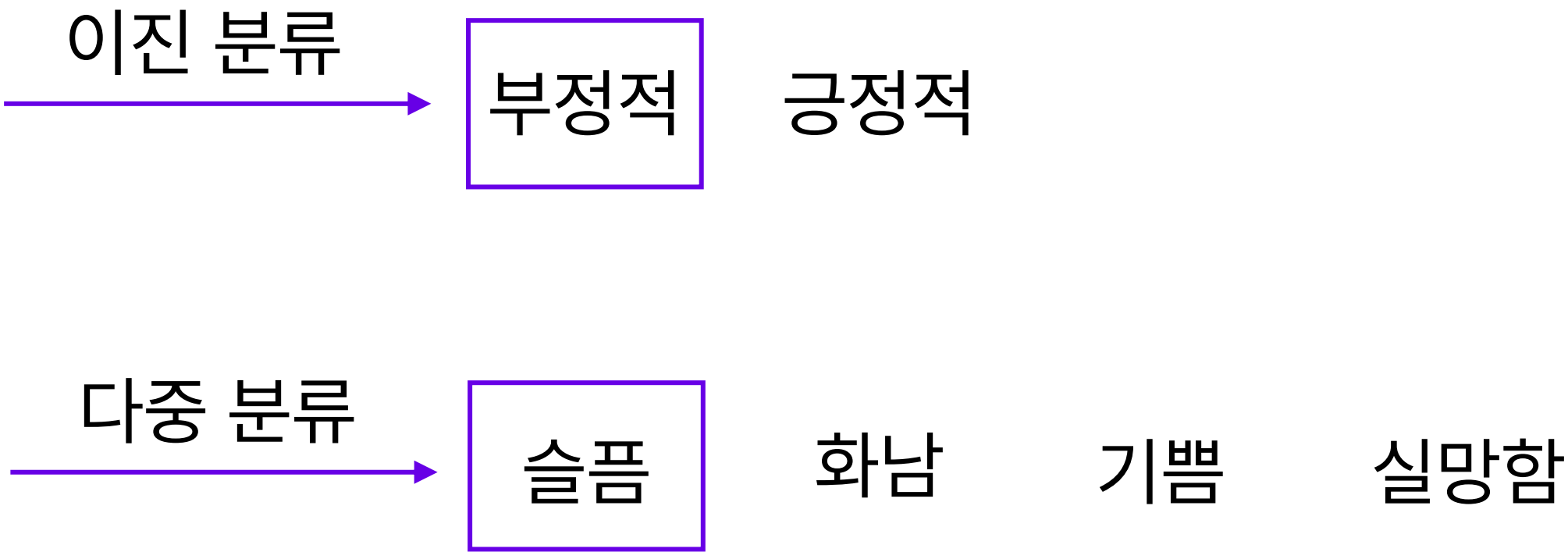
④ 다중 분류 모델

3개 이상의 종류 중 어디에 속하는지 예측하는 모델

- 종류가 3개 이상인 경우에 사용
- 원-핫 인코딩 형태로 출력하는 모델

④ 감정 이진 분류와 다중 분류

"I'm sad because it's raining."



④ 이진 분류 파이프라인 불러오기

```
binary_pipe = pipeline(  
    model="distilbert-base-uncased-finetuned-sst-2-english", device=-1  
)
```

- 모델 이름과 사용할 장치를 전달
- 여기서 장치는 CPU 혹은 GPU를 의미

④ 이진 분류 파이프라인 사용하기

```
INPUT_TEXT = "The movie I watched last night was incredibly disappointing and a complete waste of time."  
binary_pipe(INPUT_TEXT)  
# [{'label': 'NEGATIVE', 'score': 0.9998083710670471}]
```

- 불러온 파이프라인에 입력할 자연어 데이터를 전달
- 결과는 데이터를 묶는 단위인 batch 단위로 묶여서 출력됨

④ 이진 분류 파이프라인 결과

```
[{'label': 'NEGATIVE', 'score': 0.9998083710670471}]
```

- label: 어디에 속하는지 문자열
- score: 데이터가 여기에 속할 확률

= 이 데이터는 99.98%로 부정적인 문장입니다.

④ 여러 개의 데이터 동시에 입력하기

```
INPUT_TEXTS = [  
    "The weather today is fantastic, and it's putting me in a great mood.",  
    "Trying to impress my crush, I slipped on a wet floor and fell flat on my back, leaving me utterly humilia  
ted.",  
    "I recently discovered a new hobby that brings me so much joy.",  
    "The weather forecast for the weekend predicts non-stop rain, ruining any outdoor plans I had.",  
    "I successfully completed a challenging project at work, and I feel proud of my accomplishment.",  
    "I am furious about the way they treated me during the meeting.",  
]
```

- 리스트의 형태로 묶어서 파이프라인에 전달
- 입력한 순서대로 결과를 해석하여 사용

④ 여러 개의 데이터 동시에 입력하기

```
results = binary_pipe(INPUT_TEXTS)
results
```

```
[{'label': 'POSITIVE', 'score': 0.999884843826294},  
 {'label': 'NEGATIVE', 'score': 0.999647855758667},  
 {'label': 'POSITIVE', 'score': 0.9998712539672852},  
 {'label': 'NEGATIVE', 'score': 0.9997398257255554},  
 {'label': 'POSITIVE', 'score': 0.9998818635940552},  
 {'label': 'NEGATIVE', 'score': 0.9991269707679749}]
```

- 리스트의 형태로 묶어서 파이프라인에 전달
- 입력한 순서대로 결과를 해석하여 사용

④ 다중 분류 모델 불러오기

```
mluti_pipe = pipeline(model="SamLowe/roberta-base-go_emotions", device=0)
results = mluti_pipe(INPUT_TEXTS)
```

- 불러오고 사용하는 방법은 이진 분류 모델과 동일
- 결과의 label만 여러가지가 나오도록 변경

02 텍스트 분류 모델

👉 결과 출력 함수

```
def print_results(inputs, results):  
    "여러 데이터를 보기 편한 형태로 출력하는 함수"  
    for i, value in enumerate(inputs):  
        print(f"{value} => {results[i]['label']} ({results[i]['score'] * 100:.2f}%)")
```

```
I recently discovered a new hobby that brings me so much joy. => joy (85.28%)
```

- 결과를 보기 편하게 출력해주는 함수
- 입력했던 문자열과 결과, 확률을 표시하는 함수

④ 인공지능 모델과 GPU

모든 딥러닝 연산은 행렬의 연산

- 딥러닝 모델의 연산은 행렬을 곱하거나 더하는 과정으로 구성된 계산과정
- GPU는 행렬의 연산을 빠르게 하는 장치로 딥러닝 모델 연산도 빠르게 만들 수 있음

④ 데이터 600개를 처리하는데 걸리는 시간

CPU 사용 → 14.58초 소요

GPU 사용 → 4.28초 소요

- 딥러닝 모델의 연산은 행렬을 곱하거나 더하는 과정으로 구성된 계산과정
- GPU는 행렬의 연산을 빠르게 하는 장치로 딥러닝 모델 연산도 빠르게 만들 수 있음

👉 GPU 확인하기

```
!nvidia-smi
```

```
+-----+
| NVIDIA-SMI 525.85.12      Driver Version: 525.85.12      CUDA Version: 12.0      |
+-----+-----+-----+-----+-----+-----+
| GPU  Name            Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|                                           MIG M. |
+=====+=====+=====+=====+=====+=====+
|   0   NVIDIA A100 80G...  On      | 00000000:E3:00.0 Off  |             On      |
| N/A   29C    P0     44W / 300W |              N/A    |      N/A   Default  |
|                                           |             Enabled |
+-----+-----+-----+-----+-----+-----+
...

```

- 설치된 GPU를 확인하는 명령어
- 사용하기 전에 이 명령어를 통해 먼저 GPU를 확인해야 함

② 파이프라인 장치 선택

```
binary_pipe = pipeline(  
    model="distilbert-base-uncased-finetuned-sst-2-english", device=-1  
) # GPU 사용하지 않음  
  
binary_pipe = pipeline(  
    model="distilbert-base-uncased-finetuned-sst-2-english", device=0  
) # 0번 GPU 사용
```

- device 매개변수를 통해 사용할 GPU의 번호를 전달
- GPU를 사용하지 않으려면 -1을 전달